The RexxPub Book

A Comprehensive Guide

Josep Maria Blasco

Espacio Psicoanalítico de Barcelona

May 2026

Contents

INTRODUCTION **7**

What is RexxPub? **9**

The authoring format 9

The rendering engine 9

The document classes 10

The components 10

The people 11

The Four Pipelines **13**

Serve 13

Print 13

Build 13

Render 14

When to use each pipeline 14

Quick Start **15**

Writing the source 15

Generating a PDF 16

Choosing a style 16

Choosing a size 16

Viewing in the browser 16

Other document classes 16

What comes next 17

THE UTILITIES **19**

md2pdf **21**

How it works 21

Document class detection 21

Command-line options 22
Self-contained output 22
Title extraction 23
The table of contents 23
The fix-outline trick 23
Error handling 23

The CGI Program 25
Apache configuration 25
Architecture 25
Processing flow 26
The HTML template 26
Dynamic CSS injection 27
The OptionalCall mechanism 27
The installation page 27

md2html 29
How it works 29
The template 29
The customisation file 30
The sample customisation file 31
The dynamic CSS injection 31
Command-line options 31
Error handling 32
Comparison with the CGI program 32

The Common Core 33
Coexistence with Bootstrap 33
Bootstrap overrides 33
CSS custom properties 33
Paragraph conventions 34
Protecting the Rexx Highlighter 34
Tables 34
Links 35

Footnotes 35
Page breaks 35
The horizontal scrollbar 35

The article Class 37
Page geometry 37
Typography 37
The title page 37
Parametric sizing 38
CSS architecture 38

The letter Class 39
Structural elements 39
Typography 39
Page geometry 39

The slides Class 41
Page geometry 41
Typography 41
Slide structure 41
Lists 42
No parametric sizing 42
Comparison with the old slides.css 42

The book Class 43
Page geometry and facing pages 43
Running headers 43
Chapters and parts 43
The table of contents 44
Paragraph indent suppression 45
Relationship to the article class 45

The markdown Class 47

Why Markdown 49

Why paged.js 51

The CSS Architecture **53**
Scoping to div.content 53
The @media print strategy 53
Parametric sizing with CSS custom properties 53
Design Patterns **55**
OptionalCall 55
FencedCode before Pandoc 55
Dynamic CSS injection 55
Filename as convention 56
Self-documenting documents 56
Command-Line Reference **57**
md2pdf 57
md2html 57
Dependencies and Installation **59**
Required dependencies 59
Optional dependencies 59
Verifying the installation 59
The File Inventory **61**
The bin directory 61
The css directory 61
The js directory 61
The cgi-bin directory 61

Introduction

What is RexxPub?

Publishing documents that contain Rexx source code with high-quality syntax highlighting has traditionally been a challenge. General-purpose highlighting engines like those found in GitHub or in text editors have, at best, a superficial understanding of Rexx syntax. They cannot handle the subtleties of the language: the distinction between function calls and instruction keywords, the identification of compound variables and their components, the difference between a label and an assignment, the many varieties of string and number literals, or the Rexx-specific semantics of operators and special variables.

The Rexx Highlighter, a child project of the Rexx Parser, was developed precisely to address this problem. It provides deep, semantically accurate syntax highlighting, with ANSI, HTML and LaTeX drivers, a rich palette of predefined styles, and full support for ooRexx, Classic Rexx, Executor and TUTOR-flavoured Unicode extensions.

But a highlighter alone does not make a publishing system. To produce complete, well-formatted documents — web pages, articles, slides, books, or PDF files — one needs an authoring format, a document transformation pipeline, and some way to deliver the result. RexxPub is a framework that provides all of this.

The authoring format

The authoring format is Markdown. Markdown is easy to write, easy to read, easy to learn, and easy to maintain. It does not require a specialized IDE — all of the RexxPub source files are edited with nothing more than Notepad++, a free text editor with good Unicode support. Combined with Pandoc, a universal document converter, Markdown gains footnotes, tables, citations, and many other features that make it powerful enough for technical publishing, while remaining far simpler than HTML or LaTeX.

The rendering engine

Producing print-quality output from HTML requires a rendering engine that implements the CSS Paged Media specification. RexxPub uses paged.js, a JavaScript polyfill that implements the W3C specification directly in the browser. This means that the same HTML and CSS that the web pipeline produces can be paginated in the browser simply by loading the paged.js polyfill — no separate rendering engine, no additional transformations.

For offline PDF generation, RexxPub uses pagedjs-cli, a command-line tool that runs paged.js in a headless Chromium browser. The result is the same high-quality paginated output, produced without a web server or a visible browser.

The most compelling argument for paged.js is that it implements a W3C standard. The CSS written today conforms to an open specification, and should continue to work regardless of what happens to any particular tool. As browsers gradually implement more of the CSS Paged Media specification natively, the polyfill will have less work to do — and in the limit, when browsers fully support the standard, the polyfill will no longer be needed at all, but the CSS will remain valid.

The document classes

RexxPub organises its CSS into document classes, inspired by the LaTeX convention. Each class defines a complete visual identity for a type of document:

- **article** produces paginated output that closely follows the conventions of the LaTeX `article` documentclass: DIN A4 portrait pages, Times New Roman typography, justified text with automatic hyphenation, and footnotes at the bottom of each page.
- **letter** formats formal correspondence, with structured fields for sender, recipient, date, subject, opening, and closing.
- **slides** produces landscape presentation slides, with a different page geometry and sans-serif typography.
- **book** is the most complete class, designed for long-form documents with chapters, parts, facing pages, running headers, a table of contents, and asymmetric margins for binding.

All classes share a common core of typographic conventions: the LaTeX-style paragraph indentation, the `booktabs` table style, the academic link colour, the blockquote treatment, and the code block styling that protects the Rexx Highlighter output. Each class provides three base font sizes — 10pt, 12pt, and 14pt — through a parametric system of CSS custom properties.

The document you are now reading is itself a `book.md` file, rendered by the book document class. Everything you see on the page — facing pages, running headers, the table of contents — is a live example of the class in action.

The components

RexxPub is not a single program but a collection of cooperating components:

- **The Rexx Parser** provides the deep syntactic analysis that underpins all highlighting.
- **The Rexx Highlighter** takes the Parser's output and produces richly annotated HTML (or ANSI, or LaTeX) with CSS classes for every syntactic element.
- **FencedCode.cls** is a Rexx class that processes Markdown fenced code blocks, invoking the Highlighter to replace them with highlighted HTML fragments before Pandoc sees the source.
- **Pandoc** converts the resulting Markdown (now containing highlighted HTML) into a complete HTML document.
- **paged.js** (in the browser) or **pagedjs-cli** (on the command line) paginates the HTML according to CSS Paged Media rules.
- **The CSS document classes** (article.css, letter.css, slides.css, book.css) define the visual identity for each type of document.
- **The CGI program** serves documents dynamically through Apache httpd, with a style chooser, a size chooser, and print-ready output.
- **md2html** converts directory trees of Markdown files into static HTML.
- **md2pdf** produces PDF files from the command line, orchestrating the entire pipeline from Markdown to PDF in a single invocation.

These components are described in detail in Parts II and III of this book.

The people

RexxPub is developed and maintained by the author at the Espacio Psicoanalítico de Barcelona (EPBCN), which provides the computational resources and financial support that make the project possible.

Several members of the Rexx community have made important contributions. Rony G. Flatscher developed the `rgfdark`, `rgflight`, and Vim-derived colour schemes for the Rexx Highlighter, and continues to contribute CSS styles, testing, and expert feedback. Jean Louis Faucher, the creator of Executor, inspired the development of the md2html utility, which was originally built to serve his documentation needs, and contributed the horizontal scrollbar mechanism for wide images.

This book itself was written in collaboration with Claude (Anthropic), through a process of detailed technical discussion followed by joint drafting and editing.

The Four Pipelines

Every RexxPub document, regardless of how it is delivered, passes through the same two-stage transformation at its core: first, Rexx fenced code blocks in the Markdown source are processed by FencedCode.cls, which invokes the Rexx Highlighter to produce richly annotated HTML fragments; then, Pandoc converts the resulting Markdown (now containing highlighted HTML) into a complete HTML document.

What varies between pipelines is what happens *before* and *after* this common core — how the document reaches the transformation engine, and how the result is delivered to the reader.

RexxPub provides four pipelines, each suited to a different publishing scenario.

Serve

The Serve pipeline delivers documents dynamically through a CGI program running under Apache httpd. When a reader requests a Markdown file, the CGI program processes it through FencedCode and Pandoc on the fly, wraps the result in an HTML template with Bootstrap navigation, breadcrumbs, and a sidebar, and serves the complete page.

The site `https://rexx.epbcn.com/rexx-parser/` is a live example of this pipeline. Every page on that site — documentation, tutorials, the Highlighter style gallery — is a Markdown file served dynamically by the CGI program.

The Serve pipeline also handles `.rex` and `.cls` files: when a reader requests a Rexx source file with `?view=highlight` in the query string, the CGI program highlights the entire file and serves it in the same Bootstrap template. This is how the source code listings on the site are presented.

Print

The Print pipeline adds CSS Paged Media pagination to the Serve pipeline. When the query string includes `?print=pdf`, the CGI program loads the paged.js polyfill and applies the CSS Paged Media rules, producing a print-ready layout that can be saved as PDF via the browser's print dialog.

The Print pipeline is how most RexxPub documents are previewed during development. The author writes Markdown, reloads the browser with `?print=pdf`, and immediately sees the paginated result — running headers, footnotes, page breaks, and all.

The article you are now reading, if viewed in the browser with `?print=pdf`, uses the Print pipeline. The same document, rendered offline by md2pdf, uses the Render pipeline described below. The visual result is identical because both pipelines use the same CSS and the same paged.js engine.

Build

The Build pipeline converts directory trees of Markdown files into static HTML. It is driven by the md2html utility, which walks a source directory, processes each Markdown file

through FencedCode and Pandoc, and writes the resulting HTML to an output directory, preserving the directory structure.

The Build pipeline is useful for generating documentation sites that do not need a running web server. The output is plain HTML that can be published on any hosting service, copied to a USB drive, or served by a minimal static file server.

Unlike the Serve pipeline, the Build pipeline does not include Bootstrap navigation or interactive controls — the output is the document content only, wrapped in a minimal HTML template. This makes it suitable for contexts where the reader will consume the HTML directly, without the framing of a full website.

Render

The Render pipeline produces PDF files from the command line, without requiring a web server or a visible browser. It is driven by the md2pdf utility, which orchestrates the entire transformation: Markdown to highlighted HTML (via FencedCode and Pandoc), then HTML to paginated PDF (via pagedjs-cli, which runs paged.js in a headless Chromium browser).

The Render pipeline supports all of the features of the Print pipeline — document classes, parametric sizing, style selection, footnotes, running headers, tables of contents — and adds a few that are specific to offline PDF generation: automatic document outline activation (via pikepdf), Citation Style Language support, and dependency checking.

The Render pipeline is how this book was produced. The command

```
[rexx] md2pdf book
```

transforms the Markdown source into the PDF you are reading now. The `book` document class is selected automatically because the source file is named `book.md`.

When to use each pipeline

The four pipelines are complementary, not competing. A typical RexxPub workflow might use all four:

- **Serve** for day-to-day browsing of documentation on a live site.
- **Print** for previewing paginated output during authoring, and for readers who want to print a single document from the browser.
- **Build** for publishing a frozen snapshot of a documentation tree, or for distributing documentation to environments without a web server.
- **Render** for producing final PDF files for distribution, archiving, or conference proceedings.

The common core guarantees that the highlighted Rexx code looks the same in all four pipelines. The document class CSS guarantees that the paginated output (Print and Render) is visually identical. The only differences are in the delivery mechanism and the surrounding chrome.

Quick Start

This chapter walks through the creation of a minimal document, from an empty text file to a finished PDF, in a handful of steps. The goal is to give a concrete sense of what working with RexxPub feels like before diving into the details of each utility and document class.

The examples assume that ooRexx, the Rexx Parser (which includes the Highlighter and FencedCode), Pandoc, and pagedjs-cli are all installed and available on the command line. Appendix B lists the full set of dependencies and how to install them.

Writing the source

Create a file called `article.md` with the following content:

```
::::: title-page

My First RexxPub Document
=========================

::: author
Jane Doe
:::

::: date
March 2026
:::

:::::

Introduction
============

This is a minimal article.  It has a title page, a heading,
a paragraph, and a Rexx code block.

A simple program
----------------

~~~rexx
/* greet.rex -- a tiny Rexx program */
Parse Arg name
If name = "" Then name = "world"
Say "Hello," name"!"
~~~
```

This is ordinary Pandoc Markdown. The `::::: title-page` block defines the title page using fenced divs. The `~~~rexx` block marks a Rexx fenced code block that will be syntax-highlighted automatically.

Generating a PDF

From the command line, run:

```
[rexx] md2pdf article
```

This single command performs the entire pipeline: FencedCode processes the Rexx block through the Highlighter, Pandoc converts the Markdown to HTML, and pagedjs-cli paginates the result into a PDF file. The document class is `article`, inferred from the filename.

The output is a file called `article.pdf` — a DIN A4 document with the title page, the heading, the paragraph, and the Rexx code block rendered with full syntax highlighting in the default dark style.

Choosing a style

To use a different highlighting style, pass the `--style` option:

```
[rexx] md2pdf --style light article
```

The available styles include `dark` (the default), `light`, `rgfdark`, `rgflight`, and a large collection of Vim-derived colour schemes. The full list is available in the Rexx Highlighter documentation.

Choosing a size

To change the base font size, pass the `--size` option:

```
[rexx] md2pdf --size 10 article
```

This produces a 10pt version with wider margins, keeping the line length comfortable. The available sizes are 10, 12 (the default), and 14.

Viewing in the browser

If the CGI pipeline is installed (see Chapter 5), the same document can be viewed in the browser by navigating to its URL. Appending `?print=pdf` activates paged.js pagination, producing the same paginated layout as the Render pipeline.

The style and size choosers in the browser's toolbar let the reader switch between styles and sizes interactively, without regenerating the document.

Other document classes

The same workflow applies to all document classes. To produce a letter, create a file called `letter.md` and run `md2pdf letter`. To produce slides, create `slides.md` and run `md2pdf slides`. To produce a book, create `book.md` and run `md2pdf book`. Each class has its own Markdown conventions, described in Part III of this book.

What comes next

The rest of this book covers everything in detail. Part II describes each utility: md2pdf, the CGI program, and md2html. Part III documents each document class and the common core they share. Part IV discusses the design decisions behind the framework — why Markdown, why paged.js, and how the CSS architecture works.

The Utilities

md2pdf

The simplest way to produce a finished PDF from a RexxPub document is a single command:

```
[rexx] md2pdf article
```

This invocation reads `article.md`, processes its Rexx fenced code blocks through the Highlighter, converts the Markdown to HTML via Pandoc, paginates the result with pagedjs-cli, and writes `article.pdf` — all in one step. No web server, no browser window, no intermediate files left behind.

md2pdf is the utility behind the Render pipeline described in Chapter 2. It orchestrates the same transformation that the CGI program and md2html perform, but adds the final step of PDF generation through pagedjs-cli, a command-line tool that runs paged.js in a headless Chromium browser.

How it works

The processing follows a straightforward sequence. First, md2pdf reads the source file into memory and passes it to FencedCode, which identifies Rexx fenced code blocks and replaces them with highlighted HTML fragments. The result is still Markdown — but with the Rexx blocks already rendered.

Next, md2pdf invokes Pandoc with `--citeproc` for citation processing and the `inline-footnotes.lua` filter, which converts Pandoc's footnote elements into inline `<span class="footnote">` elements that paged.js can relocate to the bottom of the page. Pandoc reads the processed Markdown from standard input and writes HTML to standard output, using ooRexx's `Address COMMAND ... With Input Using ... Output Using` mechanism to pipe the data without temporary files.

md2pdf then assembles a self-contained HTML file. Unlike the CGI program, which links to external CSS files, md2pdf inlines all stylesheets — Bootstrap, the Highlighter style, the document class CSS, and any size-specific overrides — directly into a `<style>` element. This produces a single HTML file that pagedjs-cli can render without needing access to a web server or a file server.

Finally, md2pdf invokes pagedjs-cli, which opens the HTML file in a headless Chromium browser, lets paged.js paginate the content according to the CSS Paged Media rules, and saves the result as a PDF file. The temporary HTML file is deleted after the conversion.

Document class detection

md2pdf determines the document class from the source filename. A file named `article.md` uses the `article` class; `slides.md` uses `slides`; `book.md` uses `book`; `letter.md` uses `letter`. Any other filename defaults to using that filename (without extension) as the document class name, which requires a corresponding CSS file in the `css/print` directory.

This convention means that the filename is significant: renaming `article.md` to `my-paper.md` would change the document class to `my-paper`, which would fail unless a

corresponding CSS file existed. For cases where the filename does not match the desired class, the `--docclass` option provides an explicit override:

```
[rexx] md2pdf --docclass article my-paper.md
```

The `.md` extension can be omitted from the command line — md2pdf will append it automatically if the file is not found without it.

Command-line options

md2pdf accepts the following options:

`--style NAME` sets the default highlighting style for Rexx fenced code blocks. The default is `dark`. The style name must correspond to an existing `rexx-NAME.css` file in the `css/flattened` directory. md2pdf uses the flattened versions of the Highlighter stylesheets (which contain all CSS in a single file) rather than the modular versions used by the CGI program.

`--size SIZE` sets the base font size. The accepted values are 10, 12, and 14; the default is 12. When a size other than 12 is selected, md2pdf loads the corresponding override stylesheet (for example, `article-10pt.css`) in addition to the main document class stylesheet.

`--docclass CLASS` overrides the document class that would normally be inferred from the filename.

`--csl NAME` sets the Citation Style Language style for bibliographic references. The default is `ieee`. The style name must correspond to a `.csl` file in the distribution's `csl` directory. This option allows documents with Pandoc citations to use any of the thousands of available CSL styles.

`-l CODE` or `--language CODE` sets the document language, which is written into the `lang` attribute of the HTML `<html>` element. The default is `en`.

`--outline N` controls the depth of the PDF document outline (bookmarks). The default is 3, which generates outline entries for `<h1>`, `<h2>`, and `<h3>`. Setting `--outline 0` suppresses the outline entirely; `--outline 6` includes all heading levels.

`--fix-outline` activates the document outline in the generated PDF so that it is visible by default when the file is opened. This requires Python and the pikepdf library. Without this option, the outline is present in the PDF but may be hidden by default in some PDF readers.

`--default OPTS` sets default options for the Rexx Highlighter, which are applied to all fenced code blocks that do not specify their own options.

`--check-deps` verifies that all external dependencies — Pandoc, Node.js, npm, and pagedjs-cli — are installed and reachable from the command line. This is useful for troubleshooting a new installation.

`-h` or `--help` displays the usage summary.

`-it` or `--itrace` enables an extended internal traceback on error, useful for debugging problems in FencedCode or the Highlighter.

Self-contained output

A key design decision in md2pdf is that the intermediate HTML file is entirely self-contained. All CSS — Bootstrap, the Highlighter style, the document class, and any size overrides — is

concatenated and injected into a single `<style>` element. There are no external `<link>` references.

This means that pagedjs-cli does not need to resolve any URLs. The HTML file can be rendered from a temporary directory, without a web server, without file: protocol permissions, and without worrying about relative paths. It is also why md2pdf uses the *flattened* versions of the Highlighter stylesheets: each flattened file contains all the CSS rules for a given style in a single file, ready to be concatenated.

Title extraction

The PDF title — which appears in the document properties and in the browser tab when the file is opened — is extracted automatically from the generated HTML. The extraction method depends on the document class.

For articles, slides, and books, md2pdf extracts the text content of the first `<h1>` element, stripping any HTML tags (including `<small>` subtitles and `<br>` line breaks). For letters, it looks for the first paragraph inside a `<div class="recipient">` and prepends "Letter to"; if no recipient is found, it falls back to the opening salutation. For unrecognised classes, the filename is used as the title.

The table of contents

When the HTML content contains a `div` with `id="toc"` (which is how the book class marks the table of contents placeholder), md2pdf inlines the `createToc.js` script into the HTML file. This script runs after paged.js has paginated the content, walks the headings in the document, and generates the table of contents with page numbers.

The fix-outline trick

PDF viewers can display a navigable outline (sometimes called "bookmarks") in a side panel. pagedjs-cli generates this outline from the heading elements, but the PDF it produces does not set the `PageMode` flag that tells the viewer to show the outline automatically when the file is opened.

The `--fix-outline` option solves this by invoking a small Python script, `fix_pdf_outline.py`, which uses the pikepdf library to set the PDF's `PageMode` to `/UseOutlines`. This is a post-processing step that modifies the PDF in place after pagedjs-cli has finished.

The script is deliberately minimal — it opens the PDF, sets one flag, and saves — because the goal is to make the smallest possible change to the already-generated file.

Error handling

md2pdf reports errors at each stage of the pipeline. If Pandoc fails, the error output is displayed. If FencedCode encounters a syntax error in a Rexx fenced code block, md2pdf reports the error with the line number relative to the original Markdown source, reconstructing the position from the block metadata. The `--itrace` option adds a full ooRexx stack trace for deeper diagnosis.

On Windows, md2pdf detects the active code page and falls back to ASCII status indicators (`[Ok]` and `[Fail!]`) when the console does not support UTF-8 — a small but practical touch for cross-platform use.

The CGI Program

The CGI program is the engine behind the Serve and Print pipelines. It is an ooRexx program that runs under Apache httpd, processing Markdown files on the fly and serving the result as complete HTML pages with Bootstrap navigation, breadcrumbs, a sidebar, and interactive controls for style and size selection.

The reference implementation serves the entire `https://rexx.epbcn.com/rexx-parser/` site. Every page on that site — documentation, tutorials, the Highlighter style gallery, even the page that documents the CGI installation itself — is a Markdown file processed dynamically by this program.

Apache configuration

The CGI program requires just three configuration elements in Apache's `httpd.conf`. An `Action` directive defines a named handler and associates it with the CGI script:

```
Action RexxCGIMarkdown "/cgi-bin/cgi.rex"
```

A `<Files>` block tells Apache to invoke this handler for all Markdown files:

```
<Files *.md>
  SetHandler RexxCGIMarkdown
</Files>
```

And a `DirectoryIndex` directive lists the filenames that serve as index pages when a directory URL is requested:

```
DirectoryIndex readme.md article.md book.md letter.md slides.md index.html
```

This means that navigating to a directory URL like `/rexx-parser/doc/rexxpub/article/` will automatically serve the `article.md` file in that directory, if one exists. The CGI program strips the index filename from the URI to form a clean canonical URL.

The `cgi.rex` file in Apache's `cgi-bin` directory is a two-line wrapper that delegates to the main program:

```
#!rexx
Call "C:\Parser\rexx-parser\cgi\CGI.markdown.rex"
```

This indirection keeps the CGI entry point minimal and allows `CGI.markdown.rex` to be distributed as part of the Rexx Parser tree, in its natural location relative to the other source files.

Architecture

The CGI program is built on a small object-oriented framework that separates the generic CGI protocol handling from the document-specific processing logic.

The base class, `Rexx.CGI`, encapsulates the mechanics of the CGI protocol: it reads the CGI environment variables through an `HTTP.Request` object, sets up an `HTTP.Response` object for managing HTTP headers, redirects standard output to an `Array.OutputStream` (a hybrid class that collects output lines in an array while implementing the `.OutputStream` interface), and provides a three-phase execution model: setup, process, and done.

The concrete subclass, `MyCGI`, implements the `Process` method, which contains all of the document-specific logic. This separation means that a different CGI program could reuse the same base class with a completely different `Process` method, and that the base class can evolve independently of the document processing.

Processing flow

When Apache receives a request for a Markdown file, it invokes the CGI program with the file path and the request URI available through CGI environment variables. The processing then follows these steps.

The program first examines the query string for recognised parameters: `style=` to select a highlighting style, `size=` to choose a base font size (10, 12, or 14), `print=pdf` to activate paged.js pagination, and `view=highlight` to request highlighted display of `.rex` or `.cls` files. Unrecognised parameters, or parameters with invalid values, result in a 404 response — a deliberate security choice that avoids leaking information about the server's capabilities.

The style parameter is validated in two ways: first, the style name is checked against a whitelist of allowed characters (alphanumeric, plus hyphens, underscores, and periods); then, the corresponding CSS file is checked for existence on disk. This prevents both injection attacks and requests for non-existent styles.

The program then reads the source file and determines the document class from the filename. The filenames `article.md`, `book.md`, `letter.md`, and `slides.md` each map to their corresponding document class CSS; any other filename uses the generic `markdown` stylesheet.

For `.rex` and `.cls` files, the behaviour depends on the `view` parameter. Without `?view=highlight`, the file is served as plain text. With it, the entire source file is wrapped in a Rexx fenced code block and processed through FencedCode and the standard HTML template, producing a fully highlighted view of the source.

For Markdown files, the source is processed through FencedCode (which highlights Rexx fenced code blocks) and then through Pandoc (which converts the Markdown to HTML). In print mode (`?print=pdf`), the `inline-footnotes.lua` Pandoc filter is applied to convert footnotes into inline elements suitable for paged.js.

The HTML template

The CGI program uses two HTML templates, stored as ooRexx `::Resource` sections at the end of the source file. The `DisplayRexx` template is used for highlighted `.rex` and `.cls` files; the `HTML` template is used for Markdown documents.

The `HTML` template defines the page structure using Bootstrap's grid system: a full-width header, a nine-column content area with a three-column sidebar, and a footer. The template contains placeholder markers — `The RexxPub Book`, `%contents%`, `%header%`, `%footer%`, `%sidebar%`, `%contentheader%`, and several others — that the program replaces during output.

The template also contains placeholders for conditional CSS and JavaScript injection: `%filenameSpecificStyle%` for the document class stylesheet, `%sizeSpecificStyle%` for size overrides, `%printStyle%` for per-file CSS overrides, `%printJs%` for the paged.js polyfill (loaded only in print mode), and `%usedStyles%` for dynamically detected Highlighter stylesheets.

Dynamic CSS injection

One of the more elegant mechanisms in the CGI program is its handling of Highlighter stylesheets. A single document may contain Rexx fenced code blocks in multiple styles — a dark block here, a light block there. Rather than loading all available stylesheets (which would be wasteful), or requiring the author to declare which styles are used (which would be fragile), the program scans the generated HTML for `class="highlight-rexx-*"` patterns and injects `<link>` tags for only the styles actually present in the output.

This scan happens after the complete HTML has been assembled in the output buffer. The program locates the `%usedStyles%` placeholder, then walks the buffered output looking for Highlighter wrapper `<div>` elements. Each unique style name is collected, and the placeholder is replaced with the corresponding `<link>` tags.

The OptionalCall mechanism

The CGI program is designed to work out of the box with no site-specific customisation, but to accept it gracefully when it is available. This is achieved through the `OptionalCall` mechanism.

When the template contains a placeholder like `%header%`, the program attempts to call a routine named `Markdown.PageHeader`. If the routine exists (because a site-specific class has been loaded), it is called and its output is inserted into the page. If the routine does not exist, the SYNTAX condition is caught, the error code is checked to confirm it is indeed a “routine not found” error (code 43.1), and execution continues silently.

This means that the core CGI program never needs to check whether a customisation routine is available. It simply calls it, and if it is not there, nothing happens. Adding site-specific navigation, breadcrumbs, footers, or sidebars requires only defining the appropriate routines in an external class file — the CGI program itself does not need to be modified.

The reference implementation includes `rexx.epbcn.com.optional.cls`, which provides the page header (with Bootstrap navbar and logo), content header (with breadcrumb navigation, the style chooser, the size chooser, and a print button), sidebar (with context-sensitive links), and page footer (with copyright notice) for the `https://rexx.epbcn.com/` site.

The installation page

The Rexx Parser distribution includes a detailed, step-by-step installation guide for the CGI program, covering the prerequisites (Bootstrap, Pandoc, Apache httpd, ooRexx), the Apache configuration, and the verification procedure. The guide is available at `https://rexx.epbcn.com/rexx-parser/doc/highlighter/cgi/` and is itself served by the CGI program it documents.

md2html

The md2html utility drives the Build pipeline. Given a source directory containing Markdown files, it processes each one through FencedCode and Pandoc, wraps the result in an HTML template, and writes the output to a destination directory, preserving the directory structure. The result is a set of static HTML files that can be published on any hosting service, copied to a USB drive, or opened directly in a browser — no web server required.

The utility was originally developed for Jean Louis Faucher's Executor documentation, and its design reflects that origin: it is built to convert an existing tree of Markdown files into a self-contained HTML site with minimal configuration, while remaining flexible enough to accommodate different site structures through its template and customisation mechanisms.

How it works

md2html takes one or two positional arguments: a source directory (required) and a destination directory (optional, defaulting to the current directory). It scans the source directory for `.md` files using `SysFileTree`, then processes each one in sequence.

For each file, the processing follows the same stages as the other pipelines. First, the Markdown source is passed to FencedCode, which identifies Rexx fenced code blocks and replaces them with highlighted HTML fragments. Then Pandoc converts the processed Markdown to HTML, reading from standard input and writing to standard output through ooRexx's `Address COMMAND ... With Input Using ... Output Using` mechanism. The Pandoc invocation uses `--from markdown-smart+footnotes` and `--reference-location=section`.

The generated HTML is then wrapped in a template, with placeholders replaced by the appropriate content. Finally, the output is scanned for Highlighter style references, and the corresponding `<link>` tags are injected dynamically — the same technique used by the CGI program.

If the destination directory structure does not yet exist, md2html creates it automatically, mirroring the source hierarchy.

The template

The HTML template is a file called `default.md2html`. It defines the page structure — the `<head>`, the `<body>` layout, the CSS and JavaScript references — and contains placeholder markers that md2html replaces during processing.

The placeholders follow the same `%name%` convention used by the CGI program: `The RexxPub Book` for the document title (extracted from the first `<h1>` element), `%contents%` for the Pandoc-generated HTML, `%header%`, `%contentheader%`, `%footer%`, and `%sidebar%` for the optional page furniture, `%printStyle%` for a per-file CSS override, `%filenameSpecificStyle%` for the document class stylesheet, and `%usedStyles%` for the dynamically detected Highlighter stylesheets.

Two additional placeholders, `%cssbase%` and `%jsbase%`, are replaced before the template is parsed into lines. These define the base URLs for CSS and JavaScript files, and

default to the `css/` and `js/` subdirectories in the destination directory when they exist. The `--css` and `--js` options allow overriding these defaults.

Lines in the template that begin with `--` are treated as comments and stripped during loading. This allows the template file to be self-documenting without affecting the generated output.

md2html searches for the template in a well-defined sequence: first in the directory specified by `--path` (if given), then in the current directory, then in the destination directory, then in the source directory, and finally through the normal Rexx external program search order. This precedence allows project-specific templates to override the default without modifying the distribution.

The customisation file

Beyond the template, md2html supports deeper customisation through a file called `md2html.custom.rex`. This file is a Rexx program that is loaded (via `Call`) before processing begins. Its purpose is to define optional routines that md2html will invoke at specific points during the conversion.

The customisation file is found using the same search sequence as the template: `--path` directory, current directory, destination directory, source directory, and finally the Rexx external search order. The Rexx Parser distribution includes a sample copy of `md2html.custom.rex` in its `bin` directory, so the final fallback will always find a file — md2html never runs without a customisation layer, even if the user has not provided one. Unlike the template, the customisation file uses the normal Rexx `Call` mechanism rather than reading the file as text, which means that it is executed as a program and can define routines, classes, or any other Rexx constructs.

The optional routines that md2html recognises are:

`md2html.Exception` receives the filename and returns `.true` to skip the file, or `.false` (or no result) to process it. This allows certain files to be excluded from the conversion — for example, files that are meant to be included in other documents but should not be converted on their own.

`md2html.TranslatedName` receives the filename and returns a different base name for the output file, or `.nil` to use the default (the original name without the `.md` extension). This is useful when the output filenames should follow a different naming convention than the source files.

`md2html.Extension` returns the file extension for the output files. The default is `html`, but a customisation file could return `xhtml` or any other extension.

`md2html.FilenameSpecificStyle` receives the filename and returns the name of a document class CSS file to load, or `.nil` for no class-specific stylesheet. This is how md2html determines which document class CSS to apply to each file, and it works the same way as the CGI program's document class detection — the logic is simply moved into the customisation file, where it can be adapted to the project's file naming conventions.

`md2html.Header`, `md2html.ContentHeader`, `md2html.Footer`, and `md2html.Sidebar` each receive the output array and append their content to it. These routines provide the page furniture — navigation bars, breadcrumbs, footers — that the template placeholders reserve space for.

All of these routines are invoked through the same `OptionalCall` mechanism used by the CGI program: the program attempts to call the routine, and if it does not exist (SYNTAX condition 43.1), execution continues silently. This means that a minimal customisation file can define just one or two routines, and all others will simply be skipped.

The sample customisation file

The Rexx Parser distribution includes a sample `md2html.custom.rex` that illustrates how all these mechanisms work in practice. Its prologue uses package-local stems to define the configuration tables: `.Exception` maps filenames to a boolean that controls whether a file should be skipped (for example, `test_fenced_code_blocks.md` is excluded); `.TranslateFilename` maps source filenames to output names (for example, `readme.md` becomes `index`, which combines with the default `html` extension to produce `index.html`); `.FilenameSpecificStyle` maps filenames to document class names (`article.md` maps to `article`, `book.md` to `book`, `letter.md` to `letter`, `slides.md` to `slides`). All stems use a default value for unrecognised filenames.

The routine implementations are then trivially simple: each one simply looks up the stem and returns the result. `md2html.Exception` returns `.Exception[Arg(1)]`; `md2html.TranslateFilename` returns `.TranslateFilename[Arg(1)]`. This pattern — configuration in stems, logic in one-line routines — keeps the customisation file easy to read and easy to modify.

The page furniture routines (`md2html.Header`, `md2html.SideBar`, `md2html.Footer`) follow a different pattern: each reads an `::Resource` section from the same file and appends it to the output array, optionally performing simple substitutions (the header inserts the page title, the footer inserts the current year). This is the same `::Resource` technique that the CGI program uses for its HTML templates, applied at a smaller scale.

The sample file is meant to be copied and adapted. A project that uses md2html would typically place a modified copy in the source or destination directory, adjusting the exception list, the filename translations, and the page furniture to match the project's needs.

The dynamic CSS injection

After the template has been assembled with all placeholders replaced, md2html scans the output for Highlighter style references — the same post-processing hack that the CGI program uses. It locates the `%usedStyles%` placeholder line (which is still present because it was not replaced during template substitution), then walks the output lines looking for `class="highlight-rexx-*"` patterns. Each unique style name is collected, and the placeholder is replaced with the corresponding `<link>` tags pointing to the style CSS files in the `%cssbase%` directory.

This means that, as with the CGI program, the author never needs to declare which Highlighter styles a document uses. The output will contain `<link>` tags for exactly the styles that are present in the highlighted code blocks.

Command-line options

`--css BASE` or `-c BASE` sets the base URL for CSS file references in the generated HTML. This defaults to the `css/` subdirectory in the destination directory (if it exists), using a

`file:///` URL for local access. For a site intended to be served over HTTP, this should be set to the appropriate URL path.

`--js BASE` or `-j BASE` sets the base URL for JavaScript file references, with the same defaulting logic as `--css`.

`--path DIR` or `-p DIR` specifies an additional directory to search for the template and customisation files, with highest priority. This is useful when several projects share the same template and customisation but are built from different source directories.

`--default ATTRS` sets default attributes for Rexx fenced code blocks, which are passed to FencedCode. This works the same way as the corresponding option in md2pdf.

`--continue` tells md2html to continue processing when a fenced code block contains a syntax error, rather than aborting. This is useful during development, when some code blocks may be incomplete or intentionally erroneous.

`-h` or `--help` displays the usage summary.

`-it` or `--itrace` enables extended internal tracebacks on error.

Error handling

md2html verifies that Pandoc is available before processing begins, aborting with a clear message if it is not found. During processing, FencedCode errors are caught and reported with reconstructed line numbers, using the same logic as md2pdf. The `--itrace` option adds a full ooRexx stack trace for deeper diagnosis.

The `--continue` option is particularly useful for md2html because it processes entire directory trees. Without it, a single syntax error in one fenced code block would abort the entire conversion. With it, the error is reported but processing continues with the remaining files.

After all files have been processed, md2html prints a summary line with the number of files processed and the elapsed time.

Comparison with the CGI program

md2html and the CGI program share the same core architecture: both read Markdown, process it through FencedCode and Pandoc, wrap the result in an HTML template with placeholder substitution, inject CSS dynamically, and support the OptionalCall mechanism for site-specific customisation. The differences lie in their operating context and their defaults.

The CGI program runs on every request, processes a single file, and produces output that is served immediately and never stored. It links to external CSS files on the web server and includes Bootstrap navigation, interactive controls, and the paged.js print pipeline. md2html runs once as a batch process, writes its output to disk, and produces static files that need no server. Its CSS references default to local `file:///` URLs, and it includes no interactive controls unless the customisation file adds them.

The template and customisation mechanisms are intentionally parallel, so that the same design patterns learned from one utility apply to the other. The `default.md2html` template serves the same role as the `HTML` resource in the CGI program; the `md2html.custom.rex` file serves the same role as the site-specific optional class. The placeholder names and the OptionalCall routine names are consistent between the two. The Document Classes {.part}
==========

The Common Core

The four document classes — `article`, `letter`, `slides`, and `book` — share a common set of typographic conventions and CSS techniques. Understanding this shared core makes it much easier to understand the individual classes, because much of what they do is the same; the differences are largely in page geometry, text alignment, and structural elements.

This chapter describes the conventions that all classes share. The chapters that follow describe each class individually, focusing on what makes it different.

Coexistence with Bootstrap

All four classes are designed to coexist with Bootstrap 3, which provides the responsive layout for the web view: the navigation bar, the breadcrumb trail, the sidebar, and the grid system. When paged.js activates for printing (in the Print or Render pipelines), `@media print` rules hide all Bootstrap chrome — the logo, the navigation bar, the breadcrumb, the sidebar, and the footer — leaving only the document content.

This dual existence imposes an important constraint on the CSS: all typographic rules must be scoped to `div.content` to avoid interfering with Bootstrap's UI elements. For example, list rules use `div.content ul` and `div.content ol` instead of bare `ul` and `ol`, which would break Bootstrap's breadcrumb navigation. Heading rules use `div.content h1` where needed to avoid interfering with the page title. Link colours use `div.content a` to leave Bootstrap's navigation links untouched.

Bootstrap overrides

Several Bootstrap 3 defaults are explicitly overridden inside `div.content` by all document classes:

Headings receive `font-family: inherit` (to use the document's body font rather than Bootstrap's heading font stack), `line-height: 1.2` (replacing Bootstrap's `1.1`), and `break-after: avoid` to prevent headings from being stranded at the bottom of a page.

Inline code is restyled from Bootstrap's pink (`#c7254e` on `#f9f2f4`) to a neutral dark text (`#222`) on a light grey background (`#f0f0f0`), with reduced font size (0.9em) and minimal padding.

The `<pre>` element loses Bootstrap's border (`1px solid #ccc`) and border-radius, and its `word-break` is changed from `break-all` to `normal` to preserve code formatting.

Blockquotes lose Bootstrap's `border-left: 5px solid #eee` and the enlarged font size, and instead use symmetric margins matching the LaTeX `quote` environment.

CSS custom properties

All paginated classes (article, letter, book) use a common set of CSS custom properties to control their size-dependent parameters:

`--doc-font-size` sets the body text size. `--doc-line-height` sets the line spacing ratio. `--doc-footnote-size` and `--doc-fn-marker-size` control footnote text and marker sizes. `--doc-pre-size` controls the font size inside `<pre>` elements.

The default values correspond to 12pt — the LaTeX default. Loading a size-specific override stylesheet (for example, `article-10pt.css`) redefines these properties to produce a 10pt layout. The naming convention is consistent across all classes: `article-10pt.css`, `book-14pt.css`, `letter-10pt.css`, and so on.

One important caveat: `var()` references do not work reliably inside `@page` rules in all paged.js versions. For this reason, `@page` properties like margins and page number font size use literal values in the main stylesheet and are overridden with new literal values in the size-specific stylesheets.

The slides class does not use CSS custom properties because it has a single fixed size (20pt body text, 14pt code) designed for projection.

Paragraph conventions

The article, book, and letter classes each enforce a consistent paragraph style, but they do so differently.

The article and book classes follow the LaTeX convention: a 1.5em first-line indent with no vertical space between paragraphs. The first paragraph after a heading, a blockquote, a code block, or a Highlighter-output div suppresses the indent, because the preceding element already signals a new context. The `.noindent` Pandoc div class can suppress indentation explicitly, and paragraphs inside list items and definition lists never indent.

The letter class uses block letter style: no indent at all, with paragraphs separated by vertical space (0.8em bottom margin). This is the conventional style for business correspondence.

The slides class also uses block style (no indent, vertical separation), which is natural for projection slides.

Protecting the Rexx Highlighter

The Rexx Highlighter generates `<pre>` blocks inside `div.highlight-rexx-*` wrappers, with richly annotated `<span>` elements carrying `rx-*` CSS classes. The document class CSS must not interfere with this styling.

The strategy is the same in all classes: generic `<pre>` rules use the child combinator (`div.content > pre`) to target only direct children of the content area, never the `<pre>` blocks inside Highlighter wrappers. Inline `code` rules are scoped to `div.content code`, which has lower specificity than the Highlighter's class-based selectors.

This two-pronged approach means that generic code blocks (command output, configuration examples, Markdown source) receive the document class's subtle styling — a light grey background and a thin left rule — while Rexx-highlighted blocks retain their full Highlighter styling untouched.

Tables

All paginated classes use the `booktabs` convention from LaTeX: horizontal rules only (no vertical rules), with a thicker top and bottom rule (1.5pt) and a thinner rule below the header (0.75pt). Tables are centred on the page with `width: auto` instead of Bootstrap's default 100%. The slides class uses the accent colour (`#00529B`) instead of black for the rules, matching its overall colour scheme.

Links

All classes use a darker, more academic blue (`#00529B`) for links inside the content area, replacing Bootstrap's lighter blue (`#337ab7`). This is close to the default of LaTeX's `hyperref` package with `colorlinks=true`. URI links (`a.uri`) use a monospace font stack at 0.9em.

Footnotes

The article, letter, and book classes all support footnotes through paged.js's implementation of the CSS `float: footnote` mechanism. The footnote area is separated from the body text by a thin rule (0.5pt), and the footnote text uses a smaller font (`--doc-footnote-size`). Footnote markers in the body text use a superscript style at 60% of the body font size.

The slides class also supports footnotes, but uses a fixed 12pt size rather than the parametric system, and a 10pt marker.

Page breaks

All classes support the `.newpage` CSS class, which applies `break-before: page`. Any heading can force a page break by adding `{.newpage}` in the Markdown source. In print mode, the heading's top margin is suppressed so that the content begins flush at the top of the new page.

The horizontal scrollbar

All classes include the horizontal scrollbar mechanism for wide images, contributed by Jean Louis Faucher. A paragraph with the class `img-scroll` wrapping an `<img>` element will show a horizontal scrollbar when the image exceeds the content width, rather than shrinking the image to fit.

The `article` Class

The `article` class produces paginated output that closely follows the conventions of the LaTeX `article` documentclass: DIN A4 portrait pages, Times New Roman typography, justified text with automatic hyphenation, numbered headings, and footnotes at the bottom of each page.

The reference document for this class is `article.md`, a self-referencing guide that describes the class while being rendered by it. That document covers every feature in detail; this chapter provides a summary of the key design choices.

Page geometry

The default layout is DIN A4 portrait with 3cm margins on all sides. This produces a text block approximately 15cm wide, which accommodates about 66 characters per line at 12pt Times New Roman — close to the optimal line length for comfortable reading. The 10pt variant widens the margins to 3.5cm to compensate for the smaller type; the 14pt variant narrows them to 2.5cm.

Page numbers appear centred at the bottom of each page, in the same font, size, and style as the body text. The title page and blank pages suppress the page number.

Typography

The body text is set in Times New Roman at the base size (12pt by default), justified with automatic hyphenation. The `widows: 2` and `orphans: 2` properties prevent single lines from being stranded at the top or bottom of a page.

Headings follow the LaTeX hierarchy: `<h1>` at 1.4em (bold) corresponds to `\section`; `<h2>` at 1.2em (bold) to `\subsection`; `<h3>` at 1em (bold) to `\subsubsection`; `<h4>` at 1em (bold italic) to `\paragraph`. Consecutive headings collapse their top margin to avoid excessive whitespace.

The title page

The title page is built using Pandoc fenced divs. The outermost div has the class `title-page`, and it contains nested divs for the author, affiliation, address, email, phone, and date. The title itself is an `<h1>` inside the div, set at 1.7em with normal weight. An optional `<small>` element provides a subtitle in italic.

```
::::: title-page

Title of the Article <small>Optional Subtitle</small>
====================================================

::: author
Author Name
:::

::: affiliation
Institution
:::

::: date
May 2026
:::

:::::
```

All elements inside the title page are optional: a minimal title page needs only the title itself. This same fenced div structure is shared by the slides and book classes, so the same Markdown title page works across all three.

Parametric sizing

The article class supports 10pt, 12pt, and 14pt through the size-specific override stylesheets. All values are derived from the LaTeX `article` class at each point size, with minor adjustments for Times New Roman. The parametric sizing table, with the five CSS custom properties and the corresponding page margins, is documented in the `article.md` reference document.

CSS architecture

The article class is the oldest and most mature of the document classes. Its CSS architecture — Bootstrap coexistence, `div.content` scoping, child combinator for generic `<pre>` blocks, parametric custom properties — established the patterns that the letter, book, and slides classes all follow.

The `letter` Class

The `letter` class formats formal correspondence, inspired by the LaTeX `letter` documentclass. Unlike the article class, which uses title pages and section headings, the letter class uses a set of structural fenced divs that represent the conventional parts of a letter: sender, date, recipient, opening, body, closing, signature, enclosure, and carbon copy.

The reference document is `letter.md`, which is itself a letter describing the class it uses.

Structural elements

A RexxPub letter uses Pandoc fenced divs to mark the structural parts of the letter, in their conventional order:

`::::: sender` contains the sender's address block, right-aligned in European style. Markdown hard line breaks (`\`) separate the lines of the address.

`::::: date` contains the date line, also right-aligned.

`::::: recipient` contains the recipient's address, left-aligned.

`::::: opening` contains the salutation ("Dear…").

The body of the letter follows as normal Markdown paragraphs, with no special wrapper.

`::::: closing` contains the valediction ("Sincerely,").

`::::: signature` contains the sender's name, with 3em of space above for a handwritten signature.

`::::: enclosure` is an optional enclosure notice, set in a slightly smaller font (0.9em).

`::::: cc` is an optional carbon copy list, also in a smaller font.

All sections are optional. A minimal letter needs only an opening, some body text, and a closing. The CSS simply omits spacing for any section that is not present.

Typography

The letter class shares the parametric sizing system with the article class (the same five CSS custom properties, the same 10pt/12pt/14pt override stylesheets), but its typographic choices differ in several ways.

The text is ragged right instead of justified, which is the conventional style for business correspondence. There is no automatic hyphenation. Paragraphs use block letter style: no first-line indent, with 0.8em of vertical space between paragraphs. The line-height is slightly more generous (1.30 vs. 1.25 for articles) for comfortable reading.

Page geometry

The default page geometry is DIN A4 portrait with 2.5cm margins — narrower than the article class's 3cm, giving more usable space on the page. The first page has no page number; numbers appear from page two onward, centred at the bottom.

The `slides` Class

The `slides` class produces landscape presentation slides, designed for conference presentations and projected display. It is the most visually distinct of the document classes: sans-serif typography, a 16:9 page format, and a slide-per-page structure driven by `{.newpage}` headings.

The reference document is `slides.md`, which is a slide deck describing the class it uses.

Page geometry

The slide size is 254mm × 142.875mm — an exact 16:9 aspect ratio that is compatible with Full HD projection (1920 × 1080). Margins are compact (1.5cm top/bottom, 1.8cm sides) to maximise the usable area. Slide numbers appear in the bottom-right corner in a small grey font (10pt), hidden on the title slide.

Typography

The body text is set in Helvetica/Arial at 20pt, left-aligned without justification or hyphenation — all choices optimised for projection readability at a distance. There is no first-line indent; paragraphs are separated by vertical space.

Headings use the slide's accent colour (`#00529B`). The `<h1>` slide title is set at 1.6em with a 2pt bottom border in the accent colour, creating a clear visual separator between the title and the content. Sub-headings (`<h2>` and `<h3>`) are smaller and use neutral colours.

Code blocks use a fixed 14pt size — smaller than the body text, but large enough for projection.

Slide structure

The slides class uses a heading-driven structure where each `<h1>` with `{.newpage}` starts a new slide:

```
Slide Title {.newpage}
======================

Slide content goes here.
```

Section dividers use both `.part` and `.newpage` on an `<h1>`, producing a centred title without the underline bar — useful for separating major sections of the presentation:

```
Section Name {.part .newpage}
=============================
```

The title page uses the same `::::: title-page` fenced div structure as the article and book classes, with two additional elements specific to slides: `event` (for the conference name) and `venue` (for the location and dates). This means the same Markdown title page can

be shared between an article and its corresponding slide deck, with each class rendering it in its own style.

A closing slide can be created with `::::: closing-page`, which centres all content and removes the title underline — suitable for “Thank You” or “Questions?” slides.

Lists

Lists are the bread and butter of conference slides, and the slides class gives them more generous spacing than the article class: 0.4em between items (vs. 0.2em), and 1.5em of left margin (vs. 2.5em) to save horizontal space on the narrower landscape format.

No parametric sizing

Unlike the other paginated classes, the slides class does not use CSS custom properties or size-specific override stylesheets. There is a single fixed size optimised for projection.

Comparison with the old `slides.css`

The current slides class is a complete rewrite. The old `slides.css` was essentially the `markdown.css` web stylesheet with `@media print` rules and `@page` directives appended at the end. It inherited all of the web stylesheet’s styling (including Bootstrap’s heading sizes, code colours, and blockquote treatment) and did not have a dedicated visual identity.

The new slides class was designed from scratch for projection: a purpose-built page size, sans-serif typography, a consistent accent colour scheme, structured slide elements (title pages, section dividers, closing pages), and careful spacing tuned for readability at a distance.

The `book` Class

The `book` class is the most complete of the document classes. It is designed for long-form documents with chapters, parts, running headers, facing pages, a table of contents, and asymmetric margins for binding.

The book you are reading was produced by this class.

Page geometry and facing pages

The page size is DIN A4 portrait with asymmetric margins: 3.5cm on the inside edge (for binding) and 2.5cm on the outside edge. This is implemented using the CSS `margin-inside` and `margin-outside` properties, which paged.js translates into left/right margins that alternate on recto and verso pages.

The 10pt variant widens both margins; the 14pt variant narrows them. In both cases, the inside margin remains proportionally wider than the outside, preserving the binding allowance.

Running headers

The book class uses `string-set`, a CSS Paged Media property, to capture text from the document flow and display it in the page margins.

The book title is captured from a `<span>` with the class `book-title` inside the title-page `<h1>`. The chapter title is captured from each `<h1>` with the class `chapter`. In the Markdown source, this looks like:

```
[The RexxPub Book]{.book-title} <small>A Comprehensive Guide</small>
====================================================================
```

The `{.book-title}` attribute tells Pandoc to wrap the text in a `<span class="book-title">`, which the CSS rule `.title-page .book-title { string-set: bookTitle content(text) }` captures.

The running headers are then displayed in the `@page` margin boxes: the book title appears top-left on verso (left-hand) pages, and the chapter title appears top-right on recto (right-hand) pages, both in 9pt italic.

Page numbers appear in the outer bottom corner: bottom-left on verso pages, bottom-right on recto pages.

Chapters and parts

Chapters are marked with `{.chapter}` on an `<h1>`:

```
The Utilities {.part}
=============

md2pdf {.chapter}
======
```

Each chapter always starts on a recto page (`break-before: recto`). If the previous content ends on a recto page, a blank verso page is inserted automatically. Chapter headings are set at 1.8em bold with a 1pt bottom border.

Part dividers are marked with `{.part}` on an `<h1>`. They produce a full page with a large centred title (2.2em bold), no running headers, and no page numbers. Like chapters, parts always start on recto pages.

The title page and blank pages also suppress all running headers and page numbers through named page rules (`@page title-page`, `@page part-page`).

The table of contents

The book class supports a table of contents through a `<div>` with `id="toc"`. In the Markdown source, this is:

```
::: {#toc .toc-exclude}

## Contents {.toc-exclude}

:::
```

The `#toc` div is a placeholder that is filled by `createToc.js` before paged.js paginates the content. The `.toc-exclude` class on both the div and the heading tells the script to exclude them from the generated table of contents.

The script operates in dual mode. In the Print pipeline (the browser with `?print=pdf`), it registers as a paged.js handler and runs in the `beforeParsed` hook — the point where the DOM is complete but pagination has not yet started. In the Render pipeline (pagedjs-cli invoked by md2pdf), `Paged` is not available as a global, so the script runs immediately as an IIFE; md2pdf places it at the end of the `<body>` so that the DOM is already complete.

The core logic walks all `<h1>`, `<h2>`, and `<h3>` elements in the document, skipping headings that carry the class `toc-exclude` or that are inside a `.toc-exclude`, `.title-page`, or `.closing-page` ancestor. For each included heading, it ensures the element has an `id` attribute (generating one from the heading text if necessary), then creates a `<div class="toc-entry toc-level-N">` containing an `<a>` that links to the heading's `id`.

The page numbers are not generated by the script itself. Instead, they come from a CSS `target-counter` rule on the link's `::after` pseudo-element:

```
.toc-entry a::after {
  content: target-counter(attr(href url), page);
}
```

This is a CSS Paged Media mechanism: `target-counter` resolves the `href` attribute of the link, finds the page on which the target element landed after pagination, and inserts the page number as generated content. The page numbers are therefore always correct, even as the document is re-paginated at different sizes.

The TOC entries are styled with indentation by level: part and chapter entries (`toc-level-1`) are bold, section entries (`toc-level-2`) are indented 1.5em, subsection entries (`toc-level-3`) are indented 3em in a slightly smaller font.

The TOC pages use a named page (`@page toc-page`) that suppresses the chapter running header but preserves the book title on verso pages.

Paragraph indent suppression

The book class has the most comprehensive set of indent suppression rules. Like the article class, it suppresses the first-line indent on the first paragraph after a heading. But it also suppresses it after `<header>` elements, after `<blockquote>` elements, after `<pre>` elements, after Highlighter-output divs (`[class*="highlight-rexx-"]`), after tables, and after figures. The principle is consistent: any block-level element that visually interrupts the flow of paragraphs makes a following indent redundant, because the reader can already see that a new paragraph has begun. This is the same LaTeX convention that the article class follows, applied more thoroughly.

Relationship to the article class

The book class is, in many ways, a superset of the article class. The typographic core is the same: Times New Roman, justified text, automatic hyphenation, LaTeX heading sizes, `booktabs` tables, academic link colour, parametric sizing. The book class adds facing pages, running headers, chapters, parts, the table of contents, and asymmetric margins — features that are specific to long-form documents and that the article class does not need.

The `markdown` Class

The `markdown` class is different from the other four. It is not a paginated class and does not define `@page` rules, CSS custom properties, or any of the print-specific features. It is a web-only stylesheet used by the CGI program for Markdown files that do not match any of the recognised document class names (article, book, letter, slides).

When the CGI program serves a file called, say, `readme.md` or `installation.md`, it loads `markdown.css` instead of one of the document class stylesheets. The result is a clean, readable web page with Bootstrap navigation, but without the typographic refinements or page geometry of the document classes.

The `markdown` class defines the EPBCN site-specific styles (the navbar micro-logo, the page title colour, the sidebar font size), a set of basic heading styles (with fixed pixel sizes rather than the relative `em` sizes used by the document classes), and the web view of the title page elements (title, subtitle, author, date, event, and venue) with centred, coloured styling.

It also includes the horizontal scrollbar mechanism for wide images and basic `<pre>` block styling, but it does not scope its rules to `div.content` as carefully as the document classes do — a simplification that is acceptable for its web-only role.

For paginated output, `markdown.css` can be paired with the old `slides.css` (now superseded), which appended `@media print` rules and `@page` directives. This combination was the original approach before the current document classes were developed, and it remains available for backward compatibility. Design Decisions {.part} ==========

Why Markdown

The choice of authoring format is the most consequential decision in any publishing system. It determines what the author sees every day, how easy it is to learn the system, and how tightly the content is bound to its tools.

RexxPub uses Pandoc Markdown, and the reasons are both practical and philosophical.

The practical argument is simplicity. All of the RexxPub source files — every page on the rexx.epbcn.com site, every article, every slide deck, this book — are edited with Notepad++, a free text editor. There is no specialised IDE, no graphical layout tool, no build system beyond the RexxPub utilities themselves. The author writes plain text, and the utilities transform it into HTML or PDF. Markdown's readability as source code means that the author is always working with content, never fighting with markup.

The philosophical argument is portability. Markdown is a simple, well-understood format with many converters. If RexxPub were to disappear tomorrow, the source files would remain perfectly readable and convertible by any of the dozens of Markdown processors that exist. The content is not locked into a proprietary format or a specific tool.

An alternative that was considered and rejected is writing directly in HTML. HTML offers complete control over the output, but it is verbose, visually noisy, and painful to edit by hand. More importantly, raw HTML would mean giving up the structural conventions that Pandoc Markdown provides for free: footnotes, bibliographic references, fenced divs, and header attributes.

LaTeX was also considered. It is the gold standard for academic typesetting, and RexxPub's document classes are explicitly modelled on LaTeX conventions. But LaTeX has a steep learning curve, its error messages are notoriously opaque, and its ecosystem is large and complex. By using Markdown with CSS for typography, RexxPub gets LaTeX-quality output with a much simpler authoring experience. The trade-off is that CSS Paged Media is not as mature as LaTeX's typesetting engine — but it is improving rapidly, and the CSS is a W3C standard rather than a tool-specific format.

Why paged.js

Choosing a rendering engine for paginated output is a decision with long-term consequences. The engine determines what CSS features are available, how the output looks, and how dependent the project is on a single tool.

RexxPub uses paged.js, a JavaScript polyfill that implements the W3C CSS Paged Media specification. The choice was driven by a single overriding concern: standards compliance.

The CSS Paged Media specification is a W3C standard. The CSS that RexxPub writes today — `@page` rules, margin boxes, `string-set`, `target-counter`, `float: footnote`, `break-before: recto` — is standard CSS, defined by an open specification. It is not a proprietary format, not a tool-specific configuration language, not a set of command-line flags. It is CSS, and any tool that implements the specification should be able to render it.

This matters because publishing tools come and go. A framework that depends on a specific tool's proprietary features is hostage to that tool's continued development and availability. A framework that uses standard CSS has a much longer life expectancy, because the standard itself outlives any individual implementation.

Today, no browser fully implements the CSS Paged Media specification natively. Paged.js fills this gap as a polyfill: it intercepts the CSS, interprets the paged media rules, and lays out the pages using standard DOM manipulation. But the polyfill is not the point. The point is the CSS. As browsers gradually implement more of the specification (and they are doing so), the polyfill will have less work to do. In the limit, when browsers fully support CSS Paged Media, paged.js will become unnecessary — but the CSS will remain valid, and the documents will render correctly without any changes.

The dual-mode architecture reinforces this. In the browser (the Print pipeline), paged.js runs as a polyfill loaded alongside the document. On the command line (the Render pipeline), pagedjs-cli runs paged.js in a headless Chromium browser. Both use the same CSS and produce the same output. The author previews in the browser and generates the final PDF from the command line, with confidence that the two will match.

There are other tools that could produce paginated output from HTML and CSS: WeasyPrint, Prince, Vivliostyle, and others. Each has its strengths. What paged.js offers that the others do not is a combination of open source licensing, active development, a growing community, and — most importantly — the ability to preview the paginated result directly in the browser, in real time, as part of the normal web development workflow.

The CSS Architecture

The CSS architecture of RexxPub is shaped by an unusual constraint: every document must work both as a web page (served dynamically by the CGI program, with Bootstrap navigation) and as a paginated document (rendered by paged.js for print or PDF). The same HTML must look good in both contexts, and the same CSS must handle both modes.

This dual-mode requirement drives the three most important architectural decisions: scoping, the `@media print` strategy, and the parametric sizing system.

Scoping to `div.content`

All typographic rules in all document classes are scoped to `div.content`, the `<div>` that wraps the article content in the CGI template. This scoping ensures that the document class CSS never interferes with Bootstrap's navigation bar, breadcrumb trail, sidebar, or footer.

The scoping is most visible in the list rules. A bare `ul` or `ol` rule would affect Bootstrap's breadcrumb navigation (`ol.breadcrumb`), breaking its horizontal layout. By using `div.content ul` and `div.content ol`, the document class rules apply only to lists inside the article content.

The child combinator (`>`) adds a second level of scoping for `<pre>` blocks. Generic code blocks are styled with `div.content > pre`, which targets only direct children of the content area. Rexx Highlighter output, which lives inside `div.highlight-rexx-*` wrappers, is not a direct child of `div.content`, so the generic rules do not apply to it. This prevents the document class from interfering with the Highlighter's richly annotated styling.

The `@media print` strategy

When paged.js activates, the document switches from web mode to print mode. The CSS uses `@media print` rules to hide all Bootstrap chrome: the navigation bar, the breadcrumb, the sidebar, the footer, and any element with the class `screenonly` (which is used for the style and size choosers).

This approach has an important benefit: the transition from web to print requires no changes to the HTML. The same HTML page that the CGI program generates for web viewing can be paginated for printing simply by loading paged.js. The `?print=pdf` query parameter tells the CGI program to include the paged.js script and the document class CSS; everything else stays the same.

Parametric sizing with CSS custom properties

The parametric sizing system emerged from a specific design goal: supporting the three standard LaTeX sizes (10pt, 12pt, 14pt) without duplicating the entire stylesheet for each size.

The solution is a set of five CSS custom properties (the `--doc-*` variables) that control all size-dependent parameters. The main stylesheet uses `var()` references throughout, and a small override stylesheet (for example, `article-10pt.css`) redefines only the variables and the `@page` rules.

The `@page` rules are the one exception to the `var()` approach. In early testing, `var()` inside `@page` rules produced inconsistent results across different versions of paged.js. Rather than rely on uncertain support, the decision was made to use literal values in the `@page` rules and override them explicitly in the size-specific stylesheets. This adds a small amount of duplication (each size-specific stylesheet repeats the `@page` rules with new values), but it guarantees correct behaviour across all paged.js versions.

The initial naming convention used `--art-*` for article-specific properties, but this was changed to `--doc-*` when the same variable set was adopted by the letter, book, and slides classes. A single namespace means that a size-specific stylesheet works with any document class, and that the CSS custom properties serve as a contract between the main stylesheet and its size variants.

Design Patterns

Several recurring patterns appear throughout RexxPub's code. They were not designed as a deliberate pattern language, but emerged naturally from the constraints of the project and were then recognised and applied consistently.

OptionalCall

The OptionalCall pattern allows a utility to call customisation routines that may or may not exist, without requiring the caller to know in advance which routines are available.

The implementation is a small internal routine that attempts to call a named routine and catches the SYNTAX 43.1 condition (routine not found). If the routine exists, it is called normally. If it does not exist, the condition is trapped and execution continues silently. Any other SYNTAX condition is propagated, so genuine errors are not swallowed.

This pattern appears in the CGI program and in md2html. It is what makes the customisation files work: the CGI program calls `OptionalCall PageHeader`, and if the site-specific customisation file defines a `Markdown.PageHeader` routine, it runs; if not, the page simply has no header.

The elegance of this pattern is that it inverts the usual approach to optional behaviour. Instead of checking a flag or testing for the existence of a routine before calling it, the code simply calls the routine and handles the failure gracefully. There are no conditionals, no configuration files, no registration mechanisms — just a call and a trap.

FencedCode before Pandoc

RexxPub processes Rexx fenced code blocks *before* passing the source to Pandoc, not after. This is a deliberate architectural choice with important consequences.

The Rexx Highlighter produces richly annotated HTML with detailed CSS classes for every syntactic element. If the highlighting were done after Pandoc, the Highlighter would need to parse Pandoc's HTML output, find the code blocks, and modify them in place — a fragile and error-prone approach. By highlighting before Pandoc, FencedCode.cls replaces the Rexx fenced code blocks in the Markdown source with raw HTML blocks containing the Highlighter's output. When Pandoc encounters these raw HTML blocks, it passes them through unchanged, preserving the Highlighter's annotations exactly.

This "pre-processing" approach means that the Highlighter output never passes through Pandoc's HTML normalisation or sanitisation. The `<span>` elements, the CSS classes, the whitespace — everything is preserved exactly as the Highlighter produced it.

Dynamic CSS injection

The dynamic CSS injection mechanism (referred to as "the hack" in the source code comments) solves a chicken-and-egg problem: the HTML `<head>` must contain `<link>` tags for the Rexx Highlighter stylesheets, but the system does not know which highlighting styles are used in the document until after the document has been processed.

The solution is a two-pass approach. The template contains a `%usedStyles%` placeholder in the `<head>`. After the document body has been generated (by FencedCode and Pandoc), the program scans the output for `class="highlight-rexx-*"` patterns, collects the unique style names, and replaces the placeholder with the appropriate `<link>` tags.

This approach is used by both the CGI program and md2html. It has the practical benefit that the author never needs to specify which highlighting styles a document uses — the system discovers this automatically. It also means that a document can use multiple highlighting styles (for example, showing the same code in both dark and light themes), and the correct stylesheets will be loaded for all of them.

Filename as convention

RexxPub uses the filename of the Markdown source to determine the document class. A file named `article.md` is an article; `letter.md` is a letter; `slides.md` is a slide deck; `book.md` is a book. Any other name (such as `readme.md` or `installation.md`) uses the default `markdown` class.

This convention eliminates the need for metadata, configuration files, or command-line flags to specify the document class. The filename is the specification. The `--docclass` option on md2pdf exists as an override for cases where the convention is not convenient, but in practice it is rarely needed.

The same convention is used by the CGI program, by md2html (through the `FilenameSpecificStyle` customisation routine), and by md2pdf. All three utilities use the same mapping from filenames to document classes, which means that the same file produces the same output regardless of which pipeline is used.

The Apache `DirectoryIndex` directive extends this convention to directory URLs: when a reader requests a directory, Apache serves `readme.md`, `article.md`, `book.md`, `letter.md`, or `slides.md` (in that order), producing clean URLs without file extensions.

Self-documenting documents

The reference documents for the document classes — `article.md`, `letter.md`, `slides.md` — are written as documents of the class they describe. The article about the article class is itself an article, rendered by `article.css`. The letter about the letter class is itself a letter, formatted with `letter.css`. The slide deck about the slides class is itself a slide deck, paginated by `slides.css`.

This is more than a clever trick. It serves as a living test suite: if a CSS change breaks the rendering, the reference document will show the breakage immediately, because the document is its own test case. It also provides the most authentic possible documentation, because every feature described in the text is demonstrated on the same page.

The book you are reading follows the same principle. It is a `book.md` file, rendered by `book.css`, with running headers, facing pages, and a table of contents — all features of the book class that this book documents. Appendices {.part} ==========

Command-Line Reference

This appendix summarises the command-line options for the two utilities that are invoked directly by the user: md2pdf and md2html. Full descriptions of each option, with examples, appear in the corresponding chapters of Part II.

md2pdf

Usage: `[rexx] md2pdf [options] source`

The `source` argument is a Markdown filename (with or without the `.md` extension). The output is a PDF file with the same base name.

`--check-deps` checks that all external dependencies (Pandoc, pagedjs-cli, Node.js, and optionally pikepdf) are installed and reports their versions. The program exits after the check.

`--csl file` specifies a Citation Style Language file for bibliographic references. Pandoc's `--citeproc` is always enabled; this option controls the citation format.

`--default attributes` passes default attributes to FencedCode for Rexx fenced code blocks (for example, `--default number` to enable line numbering).

`--docclass name` overrides the document class inferred from the filename. The recognised classes are `article`, `book`, `letter`, and `slides`.

`--fix-outline` runs the `fix_pdf_outline.py` post-processing script to set the PDF's `PageMode` to `/UseOutlines`, so that the document outline (bookmarks) is visible when the PDF is opened. Requires pikepdf.

`-h` or `--help` displays a usage summary.

`-it` or `--itrace` enables interactive tracing for debugging.

`-l language` or `--language language` sets the document language (passed to Pandoc as the `lang` attribute).

`--outline` generates a PDF document outline from the heading structure.

`--size N` selects the base font size. The accepted values are `10`, `12` (the default), and `14`.

`--style name` selects the Rexx Highlighter style (for example, `dark`, `light`, `rgfdark`).

md2html

Usage: `[rexx] md2html [options] source [destination]`

The `source` argument is the directory containing the Markdown files. The `destination` argument (optional, defaults to the current directory) is the directory where the HTML output will be written. Both must be existing directories.

`-c url` or `--css url` sets the base URL for CSS files. If not specified and a `css` subdirectory exists in the destination directory, the default is a `file:///` URL pointing to that subdirectory.

`--continue` continues processing when FencedCode reports an error in a file, rather than stopping.

`--default attributes` passes default attributes to FencedCode, as in md2pdf.

`-h` or `--help` displays a usage summary.

`-it` or `--itrace` enables interactive tracing for debugging.

`-j url` or `--js url` sets the base URL for JavaScript files, following the same logic as `--css`.

`-p path` or `--path path` specifies a search path for the template file (`default.md2html`) and the customisation file (`md2html.custom.rex`). The path is searched before the default locations (current directory, destination directory, source directory, and the Rexx external search order).

Dependencies and Installation

RexxPub depends on a small number of external tools. This appendix lists them and describes how to verify that they are correctly installed.

Required dependencies

ooRexx is the Rexx interpreter that runs all RexxPub utilities. Version 5.0 or later is required. ooRexx is available from `https://www.oorexx.org/` and from SourceForge. On Windows, the installer adds ooRexx to the system PATH. On Linux, the `rexx` command should be available after installation.

The Rexx Parser provides the parser, the Highlighter, and FencedCode.cls. It is distributed as a zip archive from `https://rexx.epbcn.com/rexx-parser/`. The archive should be unpacked so that the `bin` directory is accessible; the RexxPub utilities expect to find FencedCode.cls and the Highlighter in `../bin/` relative to their own location.

Pandoc is the universal document converter. Version 2.19 or later is recommended. Pandoc is available from `https://pandoc.org/installing.html`. After installation, the `pandoc` command should be available on the system PATH.

Optional dependencies

pagedjs-cli is required only for the Render pipeline (md2pdf). It runs paged.js in a headless Chromium browser. pagedjs-cli is installed via npm: `npm install -g pagedjs-cli`. It requires Node.js (version 16 or later) and downloads a compatible version of Chromium automatically on first use.

pikepdf is a Python library required only for the `--fix-outline` option in md2pdf. It is installed via pip: `pip install pikepdf`. The `fix_pdf_outline.py` script uses pikepdf to set the PDF's PageMode to `/UseOutlines`.

Apache httpd is required only for the Serve and Print pipelines (the CGI program). Apache 2.4 or later is recommended. The `mod_actions` module must be enabled for the `Action` directive.

Verifying the installation

The md2pdf utility provides a `--check-deps` option that checks all dependencies and reports their versions:

```
[rexx] md2pdf --check-deps
```

This will report the version of ooRexx, Pandoc, pagedjs-cli, Node.js, and pikepdf (if installed). Any missing dependency is flagged with a clear error message.

The File Inventory

This appendix lists the files that make up a RexxPub installation, organised by directory.

The `bin` directory

The `bin` directory contains the RexxPub utilities and their supporting files:

`md2pdf.rex` is the command-line PDF generation utility. `md2html.rex` is the static site generation utility. `FencedCode.cls` is the Rexx class that processes fenced code blocks through the Highlighter. `default.md2html` is the default HTML template for md2html. `md2html.custom.rex` is the sample customisation file for md2html. `fix_pdf_outline.py` is the Python script for PDF outline activation. `inline-footnotes.lua` is the Pandoc Lua filter that converts footnotes to `float: footnote` spans for paged.js.

The `css` directory

The `css` directory contains the document class stylesheets and the Rexx Highlighter styles:

`article.css`, `article-10pt.css`, and `article-14pt.css` are the article class stylesheet and its size variants. `book.css`, `book-10pt.css`, and `book-14pt.css` are the book class. `letter.css`, `letter-10pt.css`, and `letter-14pt.css` are the letter class. `slides.css` is the slides class (no size variants). `markdown.css` is the web-only stylesheet for generic Markdown files.

The `rexx-*.css` files are the Rexx Highlighter predefined styles: `rexx-dark.css`, `rexx-light.css`, `rexx-rgfdark.css`, `rexx-rgflight.css`, and the Vim-derived colour schemes.

The `js` directory

The `js` directory contains the JavaScript files used by the web pipelines:

`paged.polyfill.js` is the paged.js polyfill, loaded when `?print=pdf` is requested. `createToc.js` generates the table of contents for book documents. `chooser.js` handles the style chooser, size chooser, and print button in the CGI program's toolbar.

The `cgi-bin` directory

The `cgi-bin` directory contains the CGI program and its supporting classes:

`CGI.markdown.rex` is the main CGI program. `Rexx.CGI.cls` is the abstract base class for CGI programs. `HTTP.Request.cls` encapsulates the HTTP request. `HTTP.Response.cls` encapsulates the HTTP response. `Array.OutputStream.cls` provides the array output stream used by the CGI program to capture output. `rexx.epbcn.com.optional.cls` is the sample site-specific customisation file for the CGI program.